Day To dAI

Fabio Brunetto, Giovanni Indiano, Federico Rizzo

October 13, 2026

1 Introduction

Work-related stress is currently one of the main social and organizational challenges. Numerous studies show that poorly distributed workloads, tight deadlines, and inefficient planning can negatively affect employee well-being, reducing productivity, increasing absenteeism, and impacting the overall workplace environment. In highly dynamic and specialized companies such as Reply, intelligent management of weekly scheduling therefore becomes a critical factor for ensuring sustainable long-term performance.
To address this need, we developed an innovative solution based on Artificial Intelligence techniques, with the aim of minimizing employee stress resulting from the weekly assignment of tasks. Our platform combines advanced AI models with a Human-In-The-Loop (HITL) approach, in which human contribution is not replaced but integrated into evaluation and decision stages.
This hybrid architecture enables the scheduler to learn from predicted stress metrics, task characteristics, and continuous worker feedback, progressively adapting to the specific needs of each professional. The result is a more equitable, personalized, and well-being-oriented planning system, capable of reducing excessive workloads, preventing stress peaks, and improving the overall quality of the working experience.

2 Solution

The AI-based scheduling process is divided into two main phases, designed to combine machine learning, continuous adaptation, and human supervision.

1. **SETUP PHASE:** In this initial phase, the model receives a set of static and contextual information required to build its knowledge base.
   The inputs include:

   - **A review of the company Reply**, containing general information on the organizational context, team structure, standard working practices, and typical project types.
   - A training set related to the workers, composed of a collection of entries, each corresponding to an employee. For each worker, general information is provided (role, skills, seniority) along with the value of *stressTolerance*, a numerical indicator of individual stress tolerance derived from a preliminary questionnaire administered before deploying the system.
   - A set of tasks, each described through:
     - estimated completion time;
     - complexity level (*low, medium, high*);
     - expected *stressLevel*;
     - the *requiredSkills* field.

   These data allow the model to learn the initial relationships between tasks, stress, and worker characteristics.

2. **OPERATIVE PHASE:** Each week, the trained model processes a series of dynamic and updated inputs to generate an optimized schedule.
   For each worker, the system receives:

- the weekly **to-do list**;
- available working hours for the week;
- feedback on the previous week's scheduling, used to assess planning effectiveness and update clusters based on perceived stress.

These elements are processed by the AI clustering model, called **D2D Scheduler**, based on a K-means algorithm adapted to the scheduling scenario.
The D2D Scheduler attempts to generate an optimized weekly plan according to stress distribution, workflow continuity, and skill consistency. If the process does not converge to a satisfactory solution, the case is automatically forwarded to an HR Manager, who may:

(a) perform a review and re-submit the case to the D2D Scheduler;

(b) or manually define the scheduling.

If the procedure converges, the schedule is validated by HR and published.
This approach ensures robustness, flexibility, and continuous supervision through the Human-In-The-Loop paradigm.

3 RAG Pipeline Workflow

The functioning of the Retrieval-Augmented Generation (RAG) pipeline adopted in the project can be described as a sequence of integrated stages, each dedicated to preparing, indexing, and querying textual content necessary for generating accurate responses from the language model.

1. **Environment Initialization:** The pipeline begins by loading environment variables and initializing the main software dependencies. In this phase, the OpenAI client, natural language processing modules (spaCy), the SentenceTransformer embedding model, the PDF parser (PyMuPDF), and the MongoDB client are configured. Auxiliary tools for text segmentation and operational constants are also prepared.

2. **Text Extraction from PDF Documents:** After opening the PDF document, the system checks for a structured outline. If available, text is extracted chapter by chapter; otherwise, the entire document is processed as a single block. The extraction is accompanied by light normalization to remove typographical artifacts and standardize the text.

3. **Text Chunking:** The extracted text is split into controlled-size chunks (approximately 500 characters), using a sentence-aware approach that preserves semantic boundaries. Optional padding sentences may be inserted between adjacent chunks to maintain contextual continuity. Each segment is enriched with metadata such as length and number of sentences.

4. **Database Ingestion (MongoDB):** The pipeline then establishes a secure connection with MongoDB. The collection used to store the segments is recreated to ensure a clean dataset. All chunks are inserted as separate documents, along with their metadata, and basic indexes (e.g., `chunk_id`) are created.

5. **Embedding Generation:** The selected SentenceTransformer model is loaded, and embeddings are generated for all chunks without an existing vector representation. Each segment is converted into a 768-dimensional dense vector, which is then stored back into MongoDB using batch updates to optimize performance.

6. **Vector Index Construction:** Once the collection is populated with text and embeddings, a vector search index is created using MongoDB Atlas Vector Search. The pipeline monitors the indexing status until it reaches the "READY" state, handling potential errors such as pre-existing indexes or infrastructure limits.

7. **Context Retrieval for RAG Queries:** When the user submits a request, it is first encoded into an embedding. MongoDB performs a search using the `$vectorSearch` operator to retrieve the top-K segments most relevant to the query. The retrieved chunks represent the contextual information used by the generative model.

8. **RAG Prompt Construction:** The system then builds a structured prompt combining:

   - the user's query or instruction (loaded from a dedicated file),
   - the retrieved chunks from the vector search.

   The final prompt follows a predefined template that instructs the model to rely exclusively on the provided context.

9. **Answer Generation:** The constructed prompt is sent to the OpenAI Chat Completion API. The language model processes the retrieved content and generates a final answer, always in English. The output is then saved to a file.

10. **Cleanup and Finalization:** Finally, the pipeline closes the MongoDB connection and prints status messages confirming successful completion of the procedure.